

WEB APPLICATION SECURITY ASSESSMENT

OWASP Juice Shop XSS & Injection Testing

Methodology, findings, and remediation — using Burp Suite and Chrome DevTools

About this document. This is a hands-on security assessment of OWASP Juice Shop — a deliberately vulnerable, official OWASP training application. It documents the methodology, each finding (with the exact steps taken and rationale), impact, and remediation. The goal was to understand the mechanism behind every vulnerability class, not to paste ready-made exploits.

Scope & authorization. All testing was performed exclusively on a local, authorized instance of OWASP Juice Shop (Docker, localhost:3001), self-authored labs, and PortSwigger Academy. Nothing here targets any real, third-party, or production system.

1. Executive Summary

This assessment covered Cross-Site Scripting (XSS) and related injection vulnerabilities across the client and server tiers of OWASP Juice Shop. Testing followed a repeatable **source** → **sink** methodology using Burp Suite (request interception and replay) and Chrome DevTools (DOM and console inspection).

Seven working vulnerabilities were confirmed, including a DOM-based XSS that — combined with the missing CSP and a JavaScript-readable session token — enables account takeover, and multiple **stored XSS** issues (a sanitizer bypass, a Zip-Slip file-overwrite chain, and a registration-API stored XSS that fires in the admin panel). SQL Injection exposed user credentials and an IDOR exposed other users' data. One tested vector (an HTTP header XSS) was **correctly defended** by an allowlist sanitizer — a documented negative finding.

Severity	Count	Findings
High	5	DOM XSS, Feedback stored XSS, Zip-Slip stored XSS, Registration-API stored XSS, SQL Injection
Medium	2	CSP bypass, IDOR / broken object-level authorization
Informational	2	Header XSS (defense held), API input stored unsanitized but safely rendered

2. Scope & Methodology

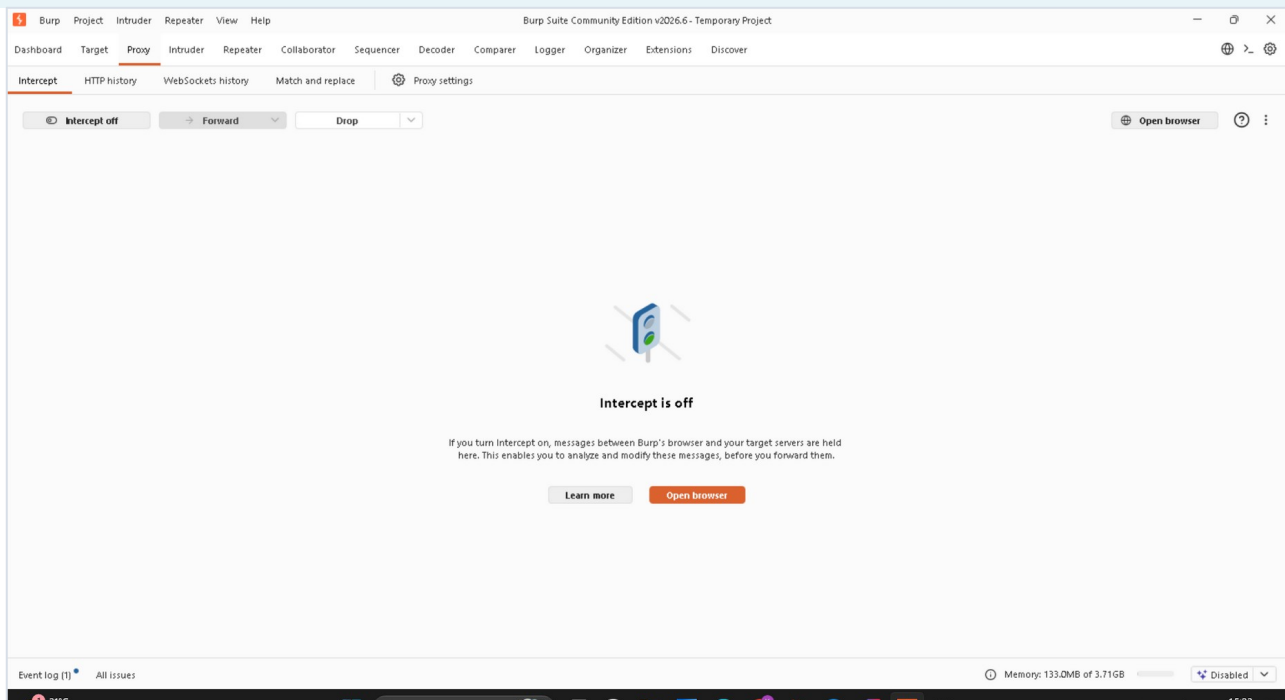
Target. OWASP Juice Shop (Angular single-page app + Node/Express backend), running locally in Docker on localhost:3001.

Tools. Burp Suite — a proxy that sits between the browser and the server, allowing requests to be inspected and edited as raw text before they reach the server (Repeater was used to modify and resend). Chrome DevTools — Elements (is the value raw HTML or escaped in the DOM?), Console (proof of execution), and Network (request/response inspection).

The source → sink loop

1. SOURCE - find every attacker-controllable input (URL params, form fields, API fields, headers, file names)
2. SINK - find where that data is rendered / written (innerHTML, bypassSecurityTrustHtml, a DB field, a file)
3. PROBE - send an inert marker (test) and read the DOM: real element (raw) or escaped text?
4. ESCALATE - if raw: , <iframe src=javascript:>, or a sanitizer/mutation bypass
5. VERIFY - prove execution cleanly with console.log(document.domain) before any alert()
6. AUTHZ - was the privilege used actually required, or is there also an access-control bug?

Which layer — network or browser? A vulnerability that stays in the browser (DOM-based XSS, mutation, clobbering) is analysed from DevTools (Console/Elements) — the Network tab is empty because the payload never reaches the server. A vulnerability that travels to the server (reflected, stored, header, API) is analysed from the Network / Burp side. Choosing the right layer is half the work.



Burp Suite proxy interface; requests were captured here and edited in Repeater.

3. Findings

Finding 1 — DOM-based XSS (search parameter)

HIGH

VULNERABILITY CLASS

DOM-based Cross-Site Scripting. The search heading writes the URL search term (?q=) into the DOM as **raw HTML** via Angular's `bypassSecurityTrustHtml` — the framework's automatic escaping is deliberately turned off at this sink.

How it works

Because escaping is disabled, the value is parsed as real HTML rather than text. A bare `<script>` inserted through an `innerHTML` path does not execute, so an event-handler element is used: a broken image fires `onerror` the moment it fails to load — a different, executing code path.

Steps taken

1. Searched for a normal term and confirmed the URL became `.../#/search?q=apple` — the source (q) is attacker-controlled.
2. Sent an inert probe `<b>apple` and inspected the DOM in **Elements** (not the rendered page): a real `<b>apple</b>` element appeared — the sink does not escape.
3. Escalated with `<img src=x onerror=console.log(document.domain)>`; the Console printed `localhost`, proving script execution. The Score Board "DOM XSS" challenge was solved.

IMPACT

Arbitrary JavaScript runs in the victim's browser. With no CSP and a session token readable by JavaScript, a crafted `?q=` link leads to session theft and full account takeover.

REMEDIATION

Remove the `bypassSecurityTrustHtml` call and rely on Angular's auto-escaping; if raw HTML is truly required, sanitize with `DOMPurify`. Add a strict CSP and store the session token in an `HttpOnly` cookie.

Finding 2 — Stored XSS via sanitizer bypass (feedback)

HIGH

VULNERABILITY CLASS

Stored XSS through a server-side sanitizer bypass. Feedback comments pass through a sanitizer, but a single-pass filter can be defeated with nested tags whose removal re-forms a new valid dangerous tag (mutation).

How it works

The filter finds and deletes a `<script>...</script>` block. After deletion, the leading `<` joins the remainder to form a valid `<iframe src="javascript:...">`. Because the filter is single-pass, it never re-checks the newly-formed tag.

Steps taken

1. Sent a plain payload first and observed the server strip the dangerous tag — confirming a server-side filter is present.
2. Submitted the nested-tag payload (request observed in Burp Repeater / the Network panel):

```
<<script>Foo</script>iframe src="javascript:alert(`xss`)">
```

3. When the comment was rendered, the alert fired and the challenge was solved. Foo is only filler for the script tag (x or 123 would work identically).

IMPACT

Persisted XSS: the payload is stored and executes for every user who views the feedback, including administrators — enabling session theft, admin actions, or content defacement.

REMEDIATION

Do not rely on blacklist/single-pass filtering. Apply output encoding at render time and a tested DOM-based sanitizer (DOMPurify / sanitize-html); if a custom filter is unavoidable, run it recursively until stable. Add CSP and HttpOnly cookies as defense-in-depth.

Finding 3 — Stored XSS via Zip-Slip (subtitle overwrite)

HIGH

VULNERABILITY CLASS

Path Traversal (Zip Slip) chained into Stored XSS. Two flaws are combined: (1) a file upload that does not validate archive entry names, allowing an existing server file to be overwritten; and (2) that file (a video subtitle) being rendered raw on a page.

How it works

The hidden `/#/promotion` page plays a video whose subtitles are loaded from `owasp_promo.vtt` and written to the DOM without sanitization. Because the server does not check `../` sequences in zip entry names, a crafted archive can overwrite that subtitle file with an XSS payload.

Steps taken

1. Identified the hidden `/#/promotion` page and that its subtitle file is rendered raw.
2. Noted the complaint form's `.zip` upload does not validate file names. Built an archive whose entry path targets the real subtitle file:

```
../../frontend/dist/frontend/assets/public/videos/owasp_promo.vtt
```

```
WEBVTT
```

```
1
```

```
00:00:00.000 --> 00:00:10.000
```

```
</script><script>alert(`xss`)</script>
```

3. Uploaded the archive (request verified in Burp), confirmed the file was overwritten at `/assets/public/videos/owasp_promo.vtt`, then opened `/#/promotion` — the alert fired.

IMPACT

Server-side file overwrite plus persisted XSS. An attacker can replace legitimate content and run script in the browser of every visitor to the affected page.

REMEDIATION

Validate/normalize archive entry names on extraction; reject `../` and never write outside the target directory. Sanitize all rendered content, including subtitles and other "content" files.

Finding 4 — Stored XSS via registration API (admin panel) HIGH

VULNERABILITY CLASS

Stored XSS via a client-side validation bypass. The registration form appears to validate the email field in the browser, but posting directly to the API skips that check entirely. The stored value is rendered unsanitized in the admin user list.

How it works

Client-side validation only runs inside the form. By sending the request straight to the API, that control is never encountered — the "bypass" is really an avoidance. The admin "Registered Users" table then renders the stored email as raw HTML.

Steps taken

1. Instead of using the registration form, posted directly to the registration API:

```
POST /api/Users
{"email": "<iframe src=\"javascript:alert(`xss`)\">", "password": "xss"}
```

2. The response returned a new user object — an email containing HTML was written to the database, with no client-side validation involved.
3. Logged in as admin and opened `/#/administration`; the Registered Users table rendered the email unsanitized and the alert fired on load.

IMPACT

Persisted XSS that executes in an **administrator's** session — the highest-value context, enabling admin session theft and privileged actions.

REMEDIATION

Validate on the server, never rely on client-side checks, and output-encode every rendering location (including admin panels). Contrast with Finding 9: the same class is harmless where the client renders safely, so encoding must be applied at every sink.

Finding 5 — Content Security Policy bypass

MEDIUM

VULNERABILITY CLASS

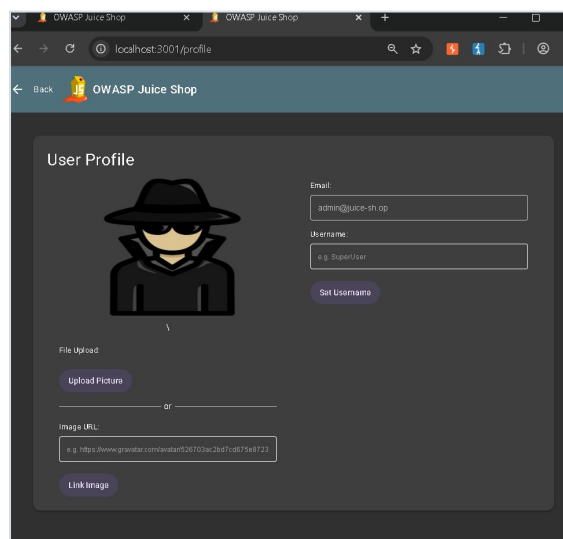
CSP bypass enabling XSS. Text entered in the profile Image URL field is copied into the page's CSP header. Because the first occurrence of a directive is authoritative, injecting a permissive rule at the start of the header loosens the policy.

How it works

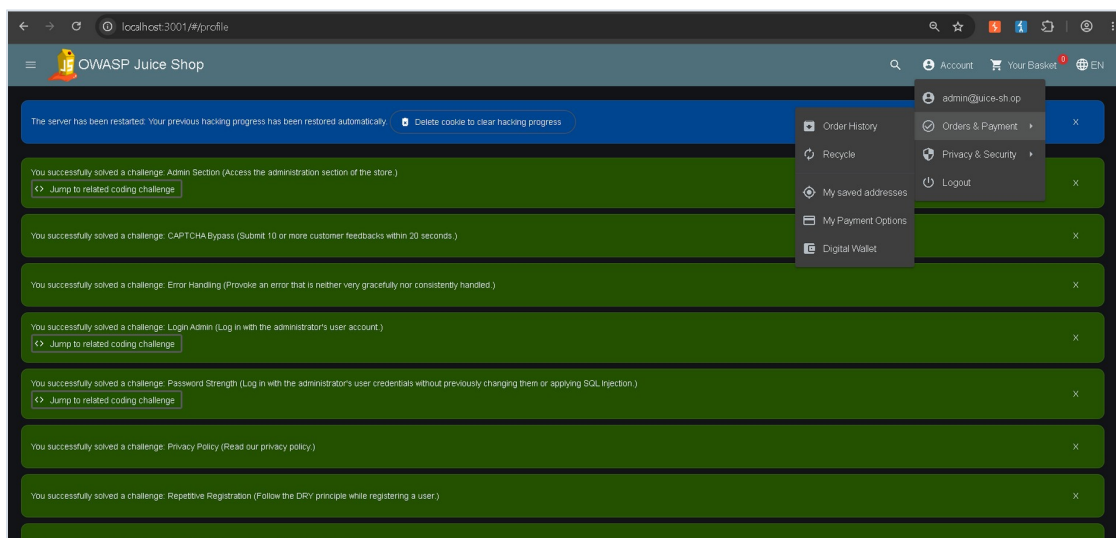
CSP is a second line of defense that stops an existing XSS from executing. Here the user controls part of the CSP header, so a permissive `script-src` can be injected ahead of the restrictive one, re-enabling inline script.

Steps taken

1. Used two profile fields together: **Username** (places the script in the DOM) and **Image URL** (rewrites the CSP).



Account menu → Privacy & Security.

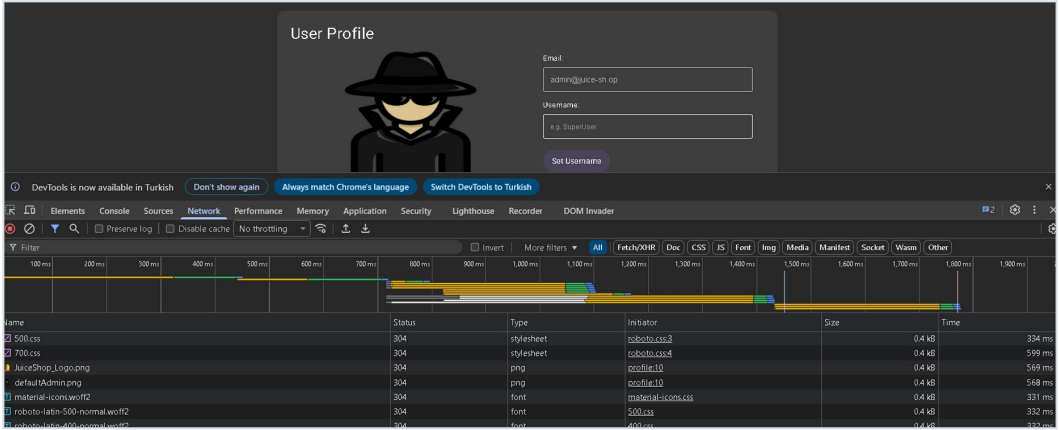


Profile page: Username and Image URL fields.

2. Placed a half-bypass payload in Username so a working `<script>` enters the DOM (not yet executed, blocked by CSP).
3. Injected a permissive CSP rule via the Image URL field:

```
https://a.png; script-src 'unsafe-inline' 'self' 'unsafe-eval'
https://code.getmdl.io http://ajax.googleapis.com
```

4. Burp's DOM Invader extension initially interfered; after **disabling DOM Invader** and reloading, the injected CSP took effect, the DOM script executed, and the alert fired — challenge solved.



DOM Invader auto-injected prototype-pollution parameters that interfered with the manual test.

IMPACT

The application's own CSP — the last barrier against XSS — can be weakened by user input, removing the protection that would otherwise contain a script-injection flaw.

REMEDIATION

Never reflect user input into a security header. Use a strict, static CSP with per-response nonces.

Finding 6 — SQL Injection (product search)

HIGH

VULNERABILITY CLASS

SQL Injection. User input in the product-search endpoint is concatenated into a SQL query without parameterization, allowing the query to be broken out of and rewritten.

How it works

A single quote unbalances the query and triggers a SQL error that leaks the query structure, confirming the injection. `UNION SELECT` then appends attacker-controlled columns, letting data from another table be returned through the product results.

Steps taken

1. Appended a single quote to the search term; the server returned a SQL error and leaked the query — input reaches SQL unsanitized.
2. Used `UNION SELECT` to break out and return the Users table's email and password hashes through the search results (Burp Repeater).
3. Also used `' OR 1=1 --` style input to bypass the login and access the admin account (Login Admin / Password Strength challenges).

IMPACT

Disclosure of all user email addresses and password hashes, and authentication bypass — a full compromise of the user store.

REMEDIATION

Use parameterized queries / ORM bindings exclusively; never concatenate user input into SQL. Store passwords with a strong, salted hash.

Finding 7 — IDOR / Broken Object-Level Authorization

MEDIUM

VULNERABILITY CLASS

Insecure Direct Object Reference (IDOR / BOLA). The server authenticates the session but does not verify that the requested object belongs to the caller.

How it works

Object access is keyed only on an id in the URL. Changing that id returns another user's object because ownership is never checked — authentication is enforced, authorization is not.

Steps taken

1. Sent my own `/rest/basket/1` request to Repeater, then changed the id to 2, 3, 4.
2. Each returned 200 OK with a different `UserId` — other users' baskets, accessed with my own token.

IMPACT

Unauthorized access to other users' data. At scale, every object keyed by a guessable id is exposed.

REMEDIATION

Enforce per-object authorization on every request (`resource.ownerId === session.userId`); do not rely on id obscurity.

1. Captured the /rest/saveLoginIp request in Burp and sent it to Repeater.

The relevant request located in Burp's HTTP history and sent to Repeater:

The screenshot shows the Burp Suite interface with the 'Request' and 'Response' tabs selected. The 'Request' tab displays a raw HTTP request, and the 'Response' tab displays a raw HTTP response. The interface includes a top toolbar with navigation and search icons, and a bottom toolbar with search and origin icons.

Request

```

1 POST /api/v1/users HTTP/1.1
2 Host: 10.10.10.10
3 Content-Type: application/json
4 Content-Length: 100
5 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/88.0.4397.100 Safari/537.36
6 Accept: application/json, text/plain, */*
7
8 {"id": "1", "username": "admin", "password": "123456", "email": "admin@10.10.10.10", "role": "admin"}
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

Response

```

1 HTTP/1.1 200 OK
2 Content-Type: application/json
3 Content-Length: 100
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/88.0.4397.100 Safari/537.36
5 Accept: application/json, text/plain, */*
6
7 {"id": "1", "username": "admin", "password": "123456", "email": "admin@10.10.10.10", "role": "admin"}
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

3. Sent the XSS payload; the `lastLoginIp` field came back empty — the payload was stripped server-side.

Payload attempt: "lastLoginIp":"" — stripped by the server.

4. Mapped the filter by testing values piece by piece (plain text passed; `<test>` was removed; `abc<test>def` returned `abcdef`; allowed tags passed but the `javascript:` scheme and event handlers were removed).

Result. This header is protected by a well-configured **allowlist** sanitizer (likely `sanitize-html`): only specific tags are permitted, `<a>` allows only `href/name/target`, and `href` accepts only `http/https/ftp/mailto`. The attack could not be turned into XSS — a documented negative finding and an example of correct defense.

Finding 9 — Unvalidated API input, safely rendered

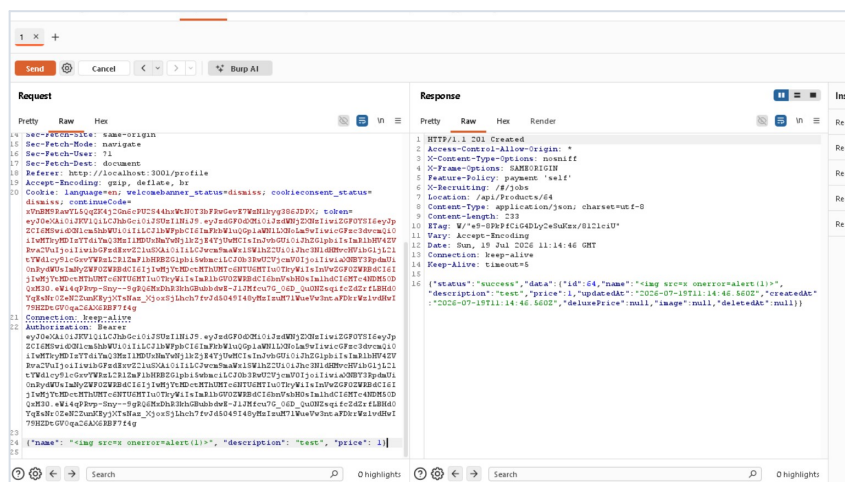
LOW

VULNERABILITY CLASS

API-only input handling (defense-in-depth). The product-creation API (/api/Products) stores the name field without sanitization, but the storefront renders it via Angular interpolation (`{{ name }}`), which auto-escapes.

Steps taken

1. Converted an authenticated request into POST /api/Products and debugged HTTP-level errors (placeholder token, Content-Length, missing blank line) until it returned 201 Created.
2. Tested field sanitization: description stripped disallowed tags (allowlist), but name stored `<img src=x onerror=...>` verbatim.
3. Verified in the storefront: the payload appeared as the product name but did **not** execute — it was rendered as escaped text.



The name field stored unsanitized; description protected but name not (inconsistent sanitization).

IMPACT

Currently not exploitable: writing unsanitized HTML to the database is a real server-side weakness, but a safe client render neutralizes it today. The flaw would become live XSS if the value were ever rendered unsafely (as in Finding 4's admin panel).

REMEDIATION

Validate and sanitize on the server for all fields, not just the visible ones — do not depend on the client's render remaining safe.

4. Findings Summary

#	Finding	Severity	Result
1	DOM-based XSS (search ?q=)	High	Confirmed — account-takeover capable (no CSP + readable token)
2	Stored XSS — feedback sanitizer bypass	High	Confirmed — nested-tag mutation defeats single-pass filter
3	Stored XSS — Zip Slip subtitle overwrite	High	Confirmed — path traversal + raw subtitle render
4	Stored XSS — registration API → admin panel	High	Confirmed — client validation bypass, fires for admin
5	CSP bypass (profile Image URL)	Medium	Confirmed — user input reflected into CSP header
6	SQL Injection (product search)	High	Confirmed — UNION dump of credentials + auth bypass
7	IDOR / BOLA (basket)	Medium	Confirmed — no per-object ownership check
8	HTTP Header XSS (True-Client-IP)	Info	Defense held — allowlist sanitizer
9	API input (/api/Products name)	Low	Stored unsanitized but safely rendered (defense-in-depth)

5. Conclusion

Across the client and server tiers, seven working vulnerabilities were confirmed and one defense was validated. The recurring lesson is that **a flaw becomes real XSS only when the server and the client agree**: the same class is harmless where the client renders safely (Finding 9) and exploitable where it does not (Finding 4). Filtering alone — especially blacklist or single-pass — and client-side validation are insufficient; the effective controls are output encoding at every sink, tested sanitization libraries, a strict CSP, parameterized queries, and per-object authorization. The highest-priority risks are the directly-exploitable DOM-based XSS and the three stored-XSS paths (feedback, Zip Slip, and the registration-API/admin-panel chain).

Educational security research on an authorized, local target (OWASP Juice Shop, Docker, localhost:3001). Self-authored labs for advanced classes (mXSS, DOM clobbering, prototype pollution, jQuery) are included in the accompanying repository.